

JAVA INTERVIEW PREPARATION

CHAPTER 46

MICROSERVICE PLATFORM
OPERATIONS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Microservice Platform Operations: Gateways, Discovery, Configuration, Service Mesh, and Safe Deployment

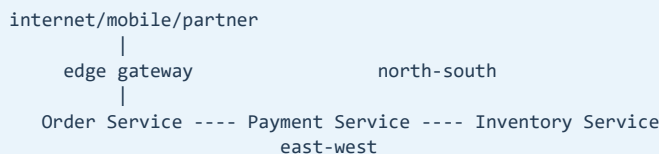
Senior / Lead Java Developer Reading Chapter

A microservice architecture needs a platform that turns changing service instances into a stable operational system. Discovery locates instances, load balancing distributes calls, gateways protect and route edge traffic, configuration supplies environment-specific policy, and deployment automation replaces versions without violating availability or compatibility.

These mechanisms do not remove application responsibility. A gateway cannot decide whether retrying a charge is safe. A service mesh cannot invent idempotency. Kubernetes can restart a process but cannot determine whether replaying its in-flight work duplicates an external effect. Senior design separates platform mechanism from business semantics and verifies how they compose.

North-South and East-West Traffic

North-south traffic crosses the external trust boundary between clients and the platform. East-west traffic moves between internal workloads.



The edge typically owns TLS termination, external authentication, coarse authorization, request-size limits, rate policy, web application firewall integration, routing, and protocol normalization. Internal communication still requires workload identity, authorization, encryption where required, deadlines, and observability.

Do not call internal traffic trusted merely because it uses a private address. Compromised workloads, configuration errors, and lateral movement remain threats. Apply least privilege at network and application boundaries.

API Gateway Responsibilities

A route combines a destination, predicates deciding which requests match, and filters running before and after proxying. Spring Cloud Gateway models these concepts directly.

```
request
-> route predicates: host + path + method
-> pre-filters: identity, limit, rewrite, trace
-> load-balanced downstream request
-> post-filters: headers, metrics, response policy
```

Useful gateway responsibilities include stable public routing, token validation or exchange, CORS for external browser clients, coarse rate limits, header sanitation, request correlation, and canary traffic selection. Centralizing these prevents inconsistent external policy.

Keep domain decisions in services. If the gateway calculates order eligibility, joins five databases, or contains every customer-specific workflow, it becomes a central business monolith and scaling bottleneck. Services must still enforce object and tenant authorization because alternative internal paths may bypass the edge.

Gateway Failure and Capacity

The gateway is on every external request path. Deploy several instances across failure domains, keep routing configuration versioned, and scale from connection count, bandwidth, TLS cost, request rate, and downstream latency. A saturated gateway can make every service appear unavailable.

Set connection acquisition, connect, response, and overall deadlines. The gateway deadline must fit the client's useful budget while allowing downstream services a shorter internal budget. Avoid automatic retries for unsafe methods unless the operation contract guarantees idempotency.

Rate limits need a stable client or tenant key. Trust identity established by authentication, not an arbitrary header supplied by the caller. Distributed limit stores introduce their own latency and availability; define fail-open or fail-closed behavior per route and threat model.

Gateways must stream large responses and uploads rather than buffering unbounded bodies. Enforce maximum sizes before expensive parsing. Protect management route inspection and mutation endpoints as privileged operations.

Header and Trust-Boundary Hygiene

Remove or overwrite headers that external clients must not control, including internal identity, tenant, role, route, and forwarding metadata. Append trusted forwarding information only after validating the immediate proxy.

```
external X-User-Id: admin    -> remove
verified token subject: 742  -> set internal identity context
untrusted X-Forwarded-For   -> ignore unless received from trusted proxy
```

Forwarded-host and scheme processing affects redirect URLs, secure cookies, and security policy. Trust forwarded headers only from known infrastructure. Normalize duplicate or ambiguous headers consistently to prevent gateway-service interpretation differences.

Trace headers and baggage are not authentication. Validate their format and size, create a new trace when invalid, and allowlist baggage keys. Do not propagate external authorization credentials to every internal service when audience-specific or workload credentials are appropriate.

Service Discovery

Instances are ephemeral: deployments replace them, autoscaling adds them, and health changes remove them. Discovery maps a logical service identity to current network endpoints.

Kubernetes commonly provides a Service name resolved by cluster DNS and backed by selected ready pods. Other platforms may use Eureka, Consul, or another registry. Spring's `DiscoveryClient` abstraction can expose registered services, and Spring Cloud Gateway can generate load-balanced routes from discovery.

```
pricing-service logical name
-> 10.2.1.7:8080
-> 10.2.3.4:8080
-> 10.2.5.9:8080
```

Discovery information is eventually updated. DNS caching, connection reuse, endpoint propagation, and client-side caches can retain a terminating instance. Graceful shutdown must mark readiness false, stop accepting new work, wait for routing propagation, and drain within the platform termination deadline.

Registration is not health. An endpoint can be discoverable but overloaded, partially broken, or unable to reach a critical dependency. Combine readiness, request-level timeouts, circuit breaking, and passive success metrics rather than assuming registry presence guarantees service.

Client-Side and Proxy Load Balancing

Client-side load balancing chooses an instance in the calling process. It avoids a separate hop and can use client-observed latency, but spreads implementation and configuration across languages and versions.

A proxy or platform service centralizes endpoint choice and transport policy. It adds infrastructure and may hide which endpoint was selected unless telemetry is propagated. Service mesh sidecars or node proxies provide a form of distributed proxying close to each workload.

Round robin is simple but ignores different request cost and in-flight work. Least-request or power-of-two choices can improve balance when instances have unequal load, provided metrics are timely. Consistent hashing can improve cache affinity but needs bounded skew and failover behavior.

Load balancing cannot fix overload. Use concurrency limits and load shedding. Continually sending traffic to every ready instance can spread a dependency-induced failure across the fleet.

Centralized Configuration

Configuration belongs outside the immutable artifact when it varies by environment: endpoints, limits, feature policy, credentials references, and observability settings. Validate required configuration at startup and expose safe effective metadata such as version or source without secret values.

Spring Cloud Config or platform-native configuration can distribute properties. The central store becomes a control-plane dependency. Cache last-known-good configuration where safe, restrict write access, review changes, and audit who changed which value.

Dynamic refresh is not automatically safer than restart. Beans may read values at different times, several properties may need atomic change, and instances can temporarily disagree. Version related properties as one configuration set and expose the active version.

```
config v17: timeout=500 ms, attempts=2, breaker-window=100
rollout: 5 percent -> 25 percent -> 100 percent
observe error and latency before wider activation
```

Feature flags require owner, purpose, default, expiry date, and unavailable-service behavior. Long-lived flags double paths and test combinations. Remove them after rollout or retirement.

Secrets and Workload Identity

Secrets are not ordinary configuration. Store them in a controlled secret manager, grant access through workload identity, rotate them, and prevent exposure through logs, environment diagnostics, heap dumps, support bundles, or images.

Mounted secret files, injected environment variables, and runtime API retrieval have different refresh and exposure behavior. Environment variables are inherited by child processes and typically require restart. Mounted files can update, but the Java client must reload them safely. Runtime retrieval introduces network and cache policy.

Rotation needs overlap. A database or API may accept old and new credentials during a window while clients rebuild pools. Monitor which credential version is used without logging the credential itself. Emergency revocation procedures should be practiced.

Prefer short-lived workload credentials to widely shared static secrets. Bind identity to the service account and audience. A secret copied to every service destroys least privilege and makes rotation a coordinated outage risk.

What a Service Mesh Provides

A service mesh places transport policy and telemetry in proxies or platform components around services. Common capabilities include mutual TLS, workload identity, traffic routing, retries, circuit policy, and request metrics.

```
Order application -> local mesh proxy === mTLS === remote proxy -> Payment application
```

The mesh can standardize transport across languages and rotate certificates without application code. It also adds another configuration plane, resource cost, failure mode, and debugging layer. A request now passes application client, proxy, network, remote proxy, and application server.

Do not enable mesh retries globally for writes. The proxy sees protocol failures but may not know business idempotency or whether the remote operation committed. Coordinate retry ownership with application libraries to prevent multiplication.

Mesh authorization can enforce workload-level policy, but object authorization remains in the application. Transport success metrics do not reveal domain rejection or stale fallback. Use the mesh for what it can know and the application for semantic truth.

Deployment Strategies

A rolling deployment gradually replaces old instances. It is efficient but requires old and new versions to coexist. Routes, schemas, events, database migrations, and caches must remain compatible during the overlap.

Blue-green deployment maintains two complete environments and switches traffic. It enables fast traffic rollback but doubles capacity and does not undo data changes. Canary deployment sends a small representative share to the new version and expands based on service-level evidence.

```
version A: 95 percent traffic
version B: 5 percent canary
compare success, latency, saturation, and domain outcomes
promote or halt using predetermined criteria
```

Canary assignment should be stable when a multi-request workflow requires one version, often through a safe identity hash or cookie. Ensure the canary sees representative tenants and workloads. Five percent of traffic that excludes the largest customers may hide the critical regression.

Database and Event Compatibility During Deployment

Use expand-and-contract migrations:

1. add new nullable column or new table
2. deploy code that reads old and new and writes compatible state
3. backfill with bounded load and progress evidence
4. switch reads after verification
5. stop old writes
6. remove old structure in a later release

Never require new code and destructive schema change to become active at the exact same instant. Old instances in a rolling deployment must continue to work. Large backfills need throttling, resumability, index planning, and replica-lag monitoring.

Events need the same coexistence discipline. Produce additive schema changes, upgrade consumers, observe adoption, then stop old variants. A rollback is not safe if the new version already emitted an event or data shape the old version cannot interpret.

Readiness, Liveness, Startup, and Shutdown

Liveness asks whether restart can recover the process. Readiness asks whether this instance should receive traffic. Startup probes or availability states prevent a slow-starting process from being killed before initialization finishes.

Do not put remote dependency checks in liveness; a shared outage can restart every service and amplify it. Readiness may include dependencies essential to the minimum offered behavior, but removing every instance can be worse than serving a documented degraded response.

Shutdown order matters:

```
mark unready
-> stop new HTTP and message intake
-> allow endpoint propagation
-> drain bounded in-flight work
-> close pools and flush telemetry
-> exit before hard termination
```

The application shutdown timeout must leave margin within the platform grace period. Message visibility, consumer-session, and load-balancer propagation time must fit the same budget.

Autoscaling and Overload

CPU scaling alone misses I/O-bound services whose threads, connections, or queues are exhausted at low CPU. Choose signals linked to the limiting resource: in-flight requests, queue age, consumer lag, connection wait, or request concurrency.

Scaling has delay. New instances need scheduling, image pull, startup, warmup, registration, and cache population. A sudden spike can overwhelm the fleet before replicas become ready. Admission control and load shedding are required even with autoscaling.

Scaling consumers above topic partitions adds no useful Kafka concurrency. Scaling application instances beyond database connection capacity divides a fixed resource and may worsen contention. Coordinate replica count, per-instance pools, and downstream quotas.

Observability and Operational Control

Expose version, configuration version, route identity, selected downstream service, and deployment cohort as bounded telemetry. Compare canary and stable outcomes. Observe gateway connection pools, route latency, rejection, discovery errors, mesh proxy resource use, certificate expiry, readiness transitions, and termination duration.

Logs and traces should show both application and proxy spans without duplicating every event. Clock skew and header propagation problems can distort timelines. Metrics remain the complete fleet-level signal when trace sampling is active.

Management endpoints that reveal routes, environment, configuration, or gateway filters are sensitive. Restrict by network and identity, audit mutation, and avoid exposing secrets through diagnostic output.

Common Senior-Level Traps

The first trap is making the gateway a domain monolith. The second is assuming service discovery means the endpoint is healthy. The third is enabling retries independently at gateway, mesh, and application layers.

Other traps include trusting forwarded identity headers from the internet, wildcard configuration refresh without atomicity, static shared secrets, liveness tied to a remote database, autoscaling beyond downstream

capacity, destructive migrations during rolling deployment, and assuming blue-green traffic rollback reverses data.

A Strong Interview Answer

I separate edge, transport, and application responsibilities. The gateway owns external routing, authentication integration, coarse limits, header sanitation, deadlines, and observability, while services retain object authorization and domain policy. Discovery maps logical names to changing ready endpoints, but request-level failure handling and graceful draining remain necessary. Configuration is versioned, validated, audited, and rolled out progressively; secrets use workload identity and rehearsed rotation. A service mesh can standardize mTLS and transport telemetry but cannot decide idempotency or fallback correctness. Deployments use coexistence-safe contracts and expand-and-contract data changes, with canary decisions based on SLO and domain outcomes. Probes, shutdown, autoscaling, and overload limits are designed from platform timing and downstream capacity.

Interview-Ready Summary

The microservice platform stabilizes dynamic workloads through gateways, discovery, configuration, identity, traffic policy, and deployment automation. Every mechanism introduces its own state, timing, and failure modes.

Safe operation requires trust-boundary hygiene, bounded deadlines, compatibility during overlap, controlled secret and configuration change, capacity-aware scaling, and graceful lifecycle integration. Platform features provide mechanisms; business correctness remains with the service.

Revision Notes

- North-south traffic crosses the external trust boundary.
- East-west traffic still requires identity, authorization, and limits.
- Gateway routes combine destinations, predicates, and ordered filters.
- Keep domain logic and object authorization inside owning services.
- The gateway is a critical capacity and availability component.
- Sanitize identity and forwarding headers at the trust boundary.
- Trace and baggage headers are not proof of authentication.
- Discovery maps logical names to changing endpoints.
- Registration or readiness does not guarantee successful handling.
- DNS and connection caches affect shutdown and endpoint propagation.
- Client-side and proxy load balancing have different ownership costs.
- Load balancing does not replace overload control.
- Version and validate configuration; audit every control-plane change.
- Dynamic refresh can leave instances temporarily inconsistent.
- Feature flags need owners, safe defaults, and expiry.
- Secrets require least privilege, rotation, and exposure controls.
- A service mesh standardizes transport, not business semantics.
- Prevent retry multiplication across gateway, mesh, and application.
- Rolling deployment requires old and new versions to coexist.
- Blue-green traffic rollback does not undo data mutations.
- Canary cohorts need representative traffic and predetermined criteria.
- Use expand-and-contract for database and event evolution.
- Liveness should not depend on shared remote services.
- Mark unready before bounded draining and termination.
- Autoscale from limiting resources and respect downstream capacity.